

Manual Técnico de Referência do Sistema

Financeiro Familiar — Documento de Versionamento e Arquitetura — 22 de agosto de 2026

Propósito deste manual: Registrar o que está implementado, identificar código oculto ou morto, e guiar futuras implementações sem perder funcionalidades existentes. Serve como bíblia técnica do projeto.

~10k Linhas de código	70+ Métodos no db.js	65+ Funções no app.js	47 Canais RPC mapeados
22 Migrations de schema	7 Duplicatas críticas	4 Funções ocultas	5 Scripts de manutenção



Capítulo 1 — Visão Geral da Arquitetura

O sistema opera em **duas modalidades simultâneas**:

- **Desktop Electron**: janela nativa, IPC via `ipcMain/ipcRenderer`, sem rede
- **Web LAN (Express)**: servidor HTTP na porta 3000, acesso pelo celular/navegador na rede local



CAMADA 1 — Renderer / Frontend

`src/renderer/app.js · src/renderer/index.html · src/renderer/landing.js`
Gerencia toda a UI: navegação, modais, gráficos, tabelas. Chama `window.api.*`



Context Bridge (contextIsolation: true) / HTTP Bearer Token



CAMADA 2 — Preload Bridge

`src/preload.js` (Electron) · `src/renderer/js/api.js` (Web)
Expõe `window.api` via `contextBridge`. No modo Web, usa Proxy com `fetch /api/rpc`



`ipcRenderer.invoke()` / `POST /api/rpc`



CAMADA 3 — Core RPC + Segurança

`src/server/core.js · src/main.js`
Helmet · CORS · Rate Limiting · Session Bearer Token · Ownership Checks · Anti-Brute-Force



Chamadas diretas de método (sem ORM)



CAMADA 4 — Database Engine

`src/database/db.js · src/database/session-repo.js · src/database/crypto-utils.js`
SQLite WAL mode · better-sqlite3 · AES-256-GCM para CPF · Sessions persistidas no DB

Perfis de Usuário e Hierarquia de Acesso

PERFIL	PROFILE_TYPE	IS_SYSTEM_ADMIN	ACESSO
ADM Geral	1	1	Acesso total: gestão de famílias, logs do servidor, backup de banco, impersonação
Responsável	2	0	Gerencia membros da família, orçamentos, pode ver lançamentos de todos
Colaborador	3	0	Acesso conforme permissões granulares definidas pelo Responsável
Caçula	4	0	Dashboard simplificado, apenas despesas rápidas (<code>renderCaculaDashboard</code>)



Capítulo 2 — Schema e Banco de Dados

Banco de dados SQLite com **modo WAL** (Write-Ahead Logging) e **Foreign Keys** habilitadas. Localização automática: `%APPDATA%/financeiro-familiar/financeiro.db` (Electron) ou `DATABASE_PATH` env (Web).

Tabelas do Sistema

TABELA	PROPÓSITO	CAMPOS-CHAVE
users	Usuários do sistema	<code>id, username, password_hash, name, first_name, last_name, email, phone, cpf (criptografado), birth_date, family_id, profile_type, is_system_admin, avatar_image, recovery_question, recovery_answer, accepted_terms_timestamp, position</code>
families	Grupos familiares	<code>id, name, quota_users (6), quota_accounts (10)</code>
user_permissions	Permissões granulares por usuário	<code>user_id, can_view_all, can_edit_all, allow_dashboard, allow_recurring, allow_accounts, allow_budget, allow_goals, allow_reports</code>
accounts	Contas e cartões	<code>id, user_id, name, type (checking/savings/wallet/credit/investment/voucher), bank, balance, color, credit_limit, closing_day, due_day, agency, account_number, is_active</code>
categories	Categorias de transação	<code>id, user_id, name, icon, color, type (income/expense)</code>
recurring_items	Despesas/receitas recorrentes	<code>id, user_id, name, type, amount, category_id, account_id, due_day, is_active, is_priority, icon, color, notes, repeat_months, start_installment, position, competence_offset</code>
transactions	Lançamentos gerados (recorrentes + avulsos)	<code>id, user_id, account_id, category_id, type, amount, description, date, is_paid, notes, recurring_item_id, is_avulso, payment_date, penalty_amount, discount_amount, competence_date, invoice_id, position</code>
invoices	Faturas mensais de cartão de crédito	<code>id, card_account_id, payment_account_id, month, year, due_date, amount, penalty_amount, discount_amount, is_paid, payment_date, is_renegotiated, renegotiation_details</code>
budgets	Límites de orçamento por categoria	<code>id, user_id, category_id, month, year, amount</code>
goals	Metas financeiras	<code>id, user_id, name, target_amount, current_amount, deadline, icon, color</code>
goal_deposits	Histórico de depósitos em metas	<code>id, goal_id, amount, note, date</code>
sessions	Sessões autenticadas (7 dias TTL)	<code>token, user_id, username, family_id, profile_type, is_system_admin, expires_at</code>
server_logs	Log de eventos do sistema	<code>id, event_type, message, family_id, created_at</code>
settings	Configurações por usuário	<code>user_id, key, value</code>

Histórico Completo de Migrations (22 passos)

#	DESCRIÇÃO	O QUE ADICIONA/CRIA
1	Coluna <code>bank</code>	accounts.bank TEXT DEFAULT 'outro'
2	Coluna <code>recurring_item_id</code>	transactions.recurring_item_id INTEGER
3	Coluna <code>is_avulso</code>	transactions.is_avulso INTEGER DEFAULT 0
3.5	Colunas de pagamento	transactions.payment_date, penalty_amount, discount_amount, competence_date
3.6	Tabela <code>invoices</code>	Faturas de cartão com UNIQUE(card_account_id, month, year)
3.7	Coluna <code>invoice_id</code>	transactions.invoice_id REFERENCES invoices(id)
4	Dados bancários	accounts.agency, accounts.account_number
4.5	Recuperação de senha	users.recovery_question, users.recovery_answer
4.6	Termos de uso	users.accepted_terms_timestamp, users.accepted_terms_version
4.7	Admin do sistema	users.is_system_admin + seed admin para 'adm'
4.8	Renegociação de fatura	invoices.is_renegotiated, invoices.renegotiation_details
4.9	Índices de performance	7 índices estratégicos no SQLite
5	Migração de tipo de conta	Recria tabela accounts para aceitar type 'voucher'
6	Reparo de foreign keys	Corrige FK órfãs em recurring_items e transactions após migration 5
7	Permissões de usuário	Cria tabela user_permissions e seed para usuários existentes
7.1	Permissões de menu	user_permissions.allow_dashboard/recurring/accounts/budget/goals/reports
8	Avatar do usuário	users.avatar_image TEXT
9	Parcelamento fixo	recurring_items.repeat_months INTEGER DEFAULT 0
10	Início de parcela	recurring_items.start_installment INTEGER DEFAULT 1
11	Tabela families	Cria tabela families com quota_users e quota_accounts
12	Família do usuário	users.family_id INTEGER REFERENCES families(id)
13	Tipo de perfil	users.profile_type INTEGER DEFAULT 2
14	Família padrão	Seed "Família Mestra" e vincula usuários existentes à família 1
15	Dados pessoais ricos	users.first_name, last_name, email, phone, cpf, birth_date
16	Migração de nomes	Split de users.name em first_name + last_name retroativamente
17	Reparo de perfis	Garante profile_type correto para todos os usuários existentes
18	Posição recorrentes	recurring_items.position INTEGER DEFAULT 0
19	Posição transações	transactions.position INTEGER DEFAULT 0
20	Posição usuários	users.position INTEGER DEFAULT 0
21	Família em logs	server_logs.family_id INTEGER DEFAULT NULL
22	Offset de competência	recurring_items.competence_offset INTEGER DEFAULT 0

⚡ Capítulo 3 — main.js — Processo Principal Electron

Ponto de entrada do aplicativo Electron. Responsável por criar a janela, inicializar o banco, subir o servidor Express LAN e registrar os IPC handlers.

src/main.js

Funções e Blocos Principais

FUNÇÃO/HANDLER	LINHA	DESCRIÇÃO
createWindow()	L11	Cria a janela BrowserWindow (1340×820, sem frame, preload configurado). Abre links externos no browser do SO.
app.whenReady()	L39	Bootstrap completo: inicializa DB, sobe Express na porta 3000, registra todos os handlers IPC dinamicamente a partir de <code>buildHandlers()</code> .
IPC Loop dinâmico	L57	Para cada canal em <code>buildHandlers(db)</code> , registra <code>ipcMain.handle(channel, fn)</code> com verificação de sessão e ownership.
backup:export (nativo)	L80	Abre diálogo nativo <code>dialog.showSaveDialog</code> para salvar o arquivo .db. Sobrescreve o handler web.
backup:exportExcel (nativo)	L89	Diálogo nativo para salvar .xlsx. Chama o handler web e salva o base64 decodificado em disco.
backup:exportJson (nativo)	L113	Diálogo nativo para salvar .json com todos os dados do usuário (<code>db.exportFullJson</code>).
backup:exportCsv (nativo)	L130	Diálogo nativo para salvar .csv de transações do período (<code>db.exportTransactionsCsv</code>).
window:minimize/maximize/close	L158	Controles de janela customizados (frame: false). Chamados pelo botão de barra de título personalizada na UI.

Dupla override de handlers de backup: Os handlers de backup têm versão *Web* em `core.js` (retorna base64 para download automático no browser) e versão *Electron nativa* em `main.js` (abre diálogo de sistema). O Electron registra seus handlers *após* o loop dinâmico, sobrescrevendo os canais de backup com a versão nativa.



Capítulo 4 — preload.js — Ponte de Segurança (Context Bridge)

Expõe a API segura para o renderer via `contextBridge.exposeInMainWorld('api', ...)`. No modo Web, o arquivo `src/renderer/js/api.js` replica a mesma interface via `fetch /api/rpc` com Bearer Token.

Mapa Completo do window.api

NAMESPACE	MÉTODO	CANAL IPC	HANDLER NO DB.JS
window.api.window	minimize()	window:minimize	mainWindow.minimize()
	maximize()	window:maximize	mainWindow.maximize/unmaximize()
	close()	window:close	mainWindow.close()
window.api.auth	login, register, getUsers, updateUser, deleteUser, deleteSelf, updatePositions, getRecoveryQuestion, resetPasswordWithAnswer, exportMyData	auth:*	login(), register(), getUsers(), updateUser(), deleteUser(), deleteUserAccount(), updateUserPositions(), getRecoveryQuestion(), resetPasswordWithAnswer(), exportMyData()
window.api.settings	get, set	settings:get/set	getSettings(), setSetting()
window.api.accounts	getAll, create, update, delete, transfer	accounts:*	getAccounts(), createAccount(), updateAccount(), deleteAccount(), transferBetweenAccounts()
window.api.categories	getAll, create, update, delete	categories:*	getCategories(), createCategory(), updateCategory(), deleteCategory()
window.api.recurring	getAll, create, update, delete, togglePriority, getMonthly, postponeInstallment, updatePositions	recurring:*	getRecurringItems(), createRecurringItem(), updateRecurringItem(), deleteRecurringItem(), toggleRecurringPriority(), getMonthlyTransactionsByRecurring(), postponeRecurringInstallment(), updateRecurringPositions()
window.api.transactions	getAll, create, update, delete, togglePaid, togglePaidWithDate, updatePositions	transactions:*	getTransactions(), createTransaction(), updateTransaction(), deleteTransaction(), toggleTransactionPaid(), toggleTransactionPaidWithDate(), updateTransactionPositions()
window.api.invoices	getMonthly, pay, renegotiate, reopen, recalculate	invoices:*	getCardInvoices(), payCardInvoice(), renegotiateCardInvoice(), reopenCardInvoice(), recalculateCardInvoice()
window.api.budgets	getAll, set	budgets:*	getBudgets(), setBudget()
window.api.goals	getAll, create, update, delete, addDeposit	goals:*	getGoals(), createGoal(), updateGoal(), deleteGoal(), addGoalDeposit()
window.api.dashboard	getSummary, getGeneralSummary, getMonthlyChart, getCategoryChart	dashboard:*	getDashboardSummary(), getGeneralDashboardSummary(), getMonthlyChart(), getCategoryChart()
window.api.reports	getCashflow, getPatrimony	reports:*	getCashflow(), getPatrimony()
window.api.backup	export, restore, exportExcel, exportJson, exportCsv	backup:*	backup(), restore (em core.js), exportFullJson(), exportTransactionsCsv()

NAMESPACE	MÉTODO	CANAL IPC	HANDLER NO DB.JS
window.api.permissions	get, update	permissions:*	getUserPermissions(), updateUserPermissions()
window.api.families	getAll, create, update, delete, checkName	families:*	getFamilies(), createFamily(), updateFamily(), deleteFamily(), checkFamilyName()
window.api.logs	get, getByFamily	server:getLogs, logs:getByFamily	getServerLogs(), getFamilyLogs()

Canais Públicos (sem autenticação)

Os seguintes canais não exigem sessão ativa: `auth:login` · `auth:register` · `auth:getRecoveryQuestion` · `auth:resetPasswordWithAnswer` · `families:checkName` · `server:getInfo`



Capítulo 5 — core.js — RPC, Segurança e Express

src/server/core.js (545 linhas)

FUNÇÃO	LINHA	DESCRIÇÃO TÉCNICA
recordLoginAttempt(username, success)	L13	Registra tentativas de login no Map <code>loginAttempts</code> . Após 5 falhas → bloqueia por 5 minutos. Em caso de sucesso, limpa o registro.
checkLoginLock(username)	L28	Retorna <code>{allowed: bool, lockTimeLeft: segundos}</code> . Chamada antes de processar cada login.
isSameFamilyUser(db, userId, session)	L37	Verifica se o userId alvo pertence à mesma família da sessão. ADM Geral e Responsável têm acesso universal.
PUBLIC_CHANNELS	L50	Set com 6 canais que não exigem autenticação (login, register, recovery, checkName, getInfo).
SENSITIVE_CHANNELS	L59	Set com 4 canais que recebem rate limiting extra (<code>authLimiter</code>): 10 tentativas por 15 min.
createOwnershipChecks(db)	L66	Retorna objeto com 47 funções de verificação de posse, uma para cada canal. Cada função recebe (session, ...args) e retorna boolean.
buildHandlers(db)	L155	Retorna objeto com 47 handlers RPC. Cada handler recebe os args do canal e chama o método correspondente no db.js. Inclui lógica de exportação Excel completa inline.
createExpressApp(db)	L451	Cria e configura o app Express: Helmet CSP, CORS (fly.dev + localhost), Rate Limit, static renderer, /health, /api/admin/backup, /api/rpc.
POST /api/rpc	L522	Endpoint principal. Valida session Bearer, executa ownershipCheck, chama handler, retorna sessionToken em respostas de login.
GET /api/admin/backup	L505	Endpoint de backup direto via header <code>x-admin-key</code> . Requer env <code>ADMIN_OPERATION_KEY</code> ≥ 16 chars. Faz download do arquivo .db.

Matriz de Permissões (createOwnershipChecks) — 47 canais

CANAL	REGRA DE ACESSO
auth:updateUser	Próprio usuário OU Responsável/ADM da mesma família
auth:deleteUser	Responsável ou ADM da mesma família
auth:deleteSelf	Somente o próprio usuário (<code>id === session.userId</code>)
auth:updatePositions	Responsável/ADM + todos os IDs devem ser da mesma família
settings:get / settings:set	Somente o próprio usuário
accounts:getAll / create	Próprio userId ou mesma família
accounts:update / delete	<code>checkAccountFamily(id, familyId)</code>
accounts:transfer	Ambas as contas devem pertencer à mesma família
categories:*	Equivalente ao padrão de accounts
recurring:*	Checks via <code>checkRecurringFamily</code> e <code>isSameFamilyUser</code>
transactions:*	Checks via <code>checkTransactionFamily</code> e <code>isSameFamilyUser</code>

CANAL	REGRA DE ACESSO
invoices:*	<code>isSameFamilyUser</code> para todos
budgets:* / goals:* / reports:* / dashboard:*	<code>isSameFamilyUser</code> para todos
permissions:update	Somente Responsável (profile_type 1 ou 2) da mesma família
families:getAll / create / update / delete	Somente ADM Geral (<code>isSystemAdmin</code> <code>===</code> <code>1</code>) ou Responsável
server:getLogs	Somente Responsável ou ADM
logs:getByFamily	Somente a própria família (<code>id</code> <code>===</code> <code>session.familyId</code>)
backup:export / restore	Somente ADM Geral ou Responsável
backup:exportExcel / Json / Csv	<code>isSameFamilyUser</code>
auth:exportMyData	Somente o próprio usuário
auth:getUsers	Qualquer usuário autenticado



Capítulo 6 — db.js — Motor de Banco de Dados

src/database/db.js (3.497 linhas) · Classe `AppDatabase`

Funções de Inicialização e Infraestrutura

MÉTODO	LINHA	DESCRIÇÃO
<code>getCardBillingCycle(closingDay, dueDay, month, year)</code>	L18	Função livre (fora da classe). Calcula período da fatura do cartão com base nos dias de fechamento e vencimento. Retorna {start, end} em ISO.
<code>constructor(dbPath?)</code>	L54	Define caminho do banco: argumento > env DATABASE_PATH > %APPDATA% > raiz do projeto.
<code>initialize()</code>	L77	Bootstrap do banco: abre, configura WAL, cria SessionRepository, roda createTables + migrateSchema + repairLegacyShiftedPayments + seedDefaultData + cleanDuplicateCategories + generateMonthlyRecurrences.
<code>createSession(user)</code>	L91	Gera token aleatório de 32 bytes (hex) e persiste via SessionRepository.
<code>getSession(token)</code>	L97	Recupera e valida expiração da sessão. Retorna {userId, username, familyId, profileType, isSystemAdmin}.
<code>deleteSession(token)</code>	L101	Remove sessão do banco (logout).
<code>migrateSchema()</code>	L106	Executa as 22 migrations em try/catch idempotentes. Inclui reconstrução de tabelas e índices.
<code>repairLegacyShiftedPayments()</code>	L538	Autocura: corrige transações recorrentes antigas com datas erradas causadas por bug de shift de mês. Roda apenas no initialize.

Autenticação e Usuários

MÉTODO	LINHA	PARÂMETROS → RETORNO	DESCRIÇÃO
<code>login(username, password)</code>	L~800	string, string → {success, user?}	Valida bcrypt hash. Em sucesso retorna dados do usuário com permissões e avatar.
<code>register(data)</code>	L~850	objeto → {success, user?}	Cria usuário com bcrypt salt 10, cria permissões padrão, vincula à família.
<code>getUsers(data)</code>	L~900	{familyId?} → array	Lista usuários. Filtra por família se familyId fornecido. Descriptografa CPF.
<code>updateUser(data)</code>	L~950	objeto → {success}	Atualiza dados do usuário. Criptografa CPF se fornecido.
<code>deleteUser(id)</code>	L~1000	int → {success}	Remove usuário e em cascata todos os seus dados (ON DELETE CASCADE).
<code>deleteUserAccount(userId)</code>	L~1050	int → {success}	Auto-exclusão: usuário deleta a si mesmo (LGPD "Direito ao Esquecimento").
<code>updateUserPositions(positions)</code>	L~1100	array → {success}	Reordena usuários na tela de família (drag-and-drop).
<code>getUserPermissions(userId)</code>	L~1150	int → objeto	Retorna permissões granulares do usuário (can_view_all, allow_* por menu).

MÉTODO	LINHA	PARÂMETROS → RETORNO	DESCRIÇÃO
updateUserPermissions(data)	L~1200	objeto → {success}	Atualiza as permissões granulares de um membro da família.
getSettings(userId)	L~1250	int → objeto	Retorna configurações do usuário como objeto chave-valor.
setSetting(userId, key, value)	L~1270	int, string, any → void	Persiste uma configuração via INSERT OR REPLACE.
getRecoveryQuestion(username)	L3241	string → {question?}	Retorna a pergunta de segurança sem revelar a resposta.
resetPasswordWithAnswer(username, answer, newPassword)	L3253	string, string, string → {success}	Valida resposta e atualiza o hash da senha se correta.
getUserById(id)	L3271	int → objeto	Busca usuário por ID com descritografia de CPF. ⚠️ DUPLICADO em L3372.

Contas (Accounts)

MÉTODO	LINHA	DESCRIÇÃO
getAccounts(userId)	L~1490	Lista contas ativas do usuário com saldo calculado. Inclui contas de membros da família se permissão can_view_all.
createAccount(data)	L~1500	Cria conta com validação de tipo (checking/savings/wallet/credit/investment/voucher).
updateAccount(data)	L1510	Atualiza dados da conta incluindo limite de crédito, dia de fechamento/vencimento.
deleteAccount(id)	L1532	Soft delete (is_active = 0) para preservar histórico de transações.
transferBetweenAccounts(data)	L1541	Cria par de transações (saída da origem, entrada no destino) em transação atômica. Tipo 'transfer'.

Categorias

MÉTODO	LINHA	DESCRIÇÃO
getCategories(userId)	L1552	Lista categorias do usuário + categorias da família (se permissão). JOIN com user para exibir dono.
createCategory(data)	L1575	Cria categoria com ícone emoji e cor hex.
updateCategory(data)	L1589	Atualiza nome, ícone e cor.
deleteCategory(id)	L1594	Remove categoria (hard delete — transações perdem a FK, ficam com category_id NULL).
cleanDuplicateCategories()	L1599	Autocura: detecta e remove categorias duplicadas por (name, user_id), preservando a de menor ID. Roda no initialize.

Recorrentes (Recurring Items)

MÉTODO	LINHA	DESCRIÇÃO
getRecurringItems(userId, type, month, year)	L1627	Lista itens recorrentes com transação do mês correspondente em JOIN LEFT. Calcula status de pagamento, referência de competência e permite filtro por tipo/mês/ano.
generateMonthlyRecurrences()	L~901	Motor de geração automática: para cada recurring_item ativo, garante que existe uma transaction para o mês atual. Calcula competence_date via

MÉTODO	LINHA	DESCRIÇÃO
		<code>competence_offset</code> . Roda no initialize e após criação.
<code>createRecurringItem(data)</code>	L1739	Cria item recorrente com suporte a parcelamento (<code>repeat_months</code>), offset de competência e geração imediata das transações do mês.
<code>updateRecurringItem(data)</code>	L1791	Atualiza o item e regenera as transações futuras não pagas. Preserva histórico de pagamentos.
<code>deleteRecurringItem(id, fromDate)</code>	L1875	Soft delete: marca <code>is_active</code> = 0. Se <code>fromDate</code> fornecido, deleta apenas transações futuras a partir daquela data.
<code>postponeRecurringInstallment({txId, itemId})</code>	L1929	Adia parcela: move a transação do mês atual para o mês seguinte, ajustando data e criando nova ocorrência.
<code>toggleRecurringPriority(id)</code>	L2021	Alterna <code>is_priority</code> (0↔1) — destacado com 🌟 na UI.
<code>updateRecurringPositions(userId, positions)</code>	L2027	Persiste nova ordem de exibição após drag-and-drop na UI.

Transações (Lançamentos)

MÉTODO	LINHA	DESCRIÇÃO
<code>getTransactions(filters)</code>	L2052	Busca com filtros: <code>userId</code> , <code>month</code> , <code>year</code> , <code>type</code> , <code>accountId</code> , <code>search</code> , <code>avulsoOnly</code> . JOIN com <code>account</code> e <code>category</code> . Retorna ordenado por <code>position/date</code> .
<code>getMonthlyTransactionsByRecurring(userId, month, year)</code>	L2097	Retorna transações do mês agrupadas por <code>recurring_item_id</code> . Usado para exibir status de pagamento nas recorrentes.
<code>createTransaction(data)</code>	L2153	Cria lançamento avulso ou vinculado. Atualiza saldo da conta se <code>is_paid</code> =1.
<code>updateTransaction(data)</code>	L2184	Atualiza transação com recalculo de saldo de conta (reverte antigo, aplica novo).
<code>deleteTransaction(idPayload)</code>	L2209	Remove transação. Se era paga, estorna saldo da conta. Suporta remoção em família (<code>familyId</code> no payload).
<code>toggleTransactionPaid(id)</code>	L2236	Alterna <code>is_paid</code> e atualiza saldo da conta em transação atômica.
<code>toggleTransactionPaidWithDate(id, paymentDate, options)</code>	L2250	Marca como pago/não pago com data real de pagamento. Suporta <code>penalty_amount</code> e <code>discount_amount</code> . Calcula status rico (antecipado com desconto, atrasado com juros, etc.).
<code>updateTransactionPositions(userId, positions)</code>	L2684	Persiste nova ordem após drag-and-drop de transações avulsas.

Faturas de Cartão de Crédito (Invoices)

MÉTODO	LINHA	DESCRIÇÃO
<code>getCardInvoices(userId, month, year)</code>	L2294	Lista faturas do mês/ano para todos os cartões do usuário. Calcula período de fechamento via <code>getCardBillingCycle</code> . Auto-cria fatura se não existir.
<code>payCardInvoice({invoiceId, paymentAccountId, paymentDate, penaltyAmount, discountAmount, userId})</code>	L2367	Marca fatura como paga: debita conta de pagamento, registra penalidade/desconto, fecha fatura. Usa transação atômica.
<code>renegotiateCardInvoice({...})</code>	L2401	Renegocia fatura em parcelas: cria item recorrente para as parcelas, registra detalhes no campo <code>renegotiation_details</code> , fecha a fatura original. Commit executado antes de gerar recorrências (fix aplicado).

MÉTODO	LINHA	DESCRIÇÃO
reopenCardInvoice({invoiceId, userId})	L2545	Reabre fatura paga: estorna débito da conta, limpa payment_date e penalty/discount. Restaura estado.
recalculateCardInvoice({invoiceId, userId})	L2659	Recalcula valor total da fatura somando todas as transações do período de fechamento vinculadas ao cartão.

Dashboard, Orçamento e Metas

MÉTODO	LINHA	DESCRIÇÃO
getDashboardSummary(userId, month, year)	L2709	Query complexa: soma receitas/despesas do mês, calcula saldo, lista contas com saldo, top categorias, transações recentes, orçamento vs. gasto. Suporta permissões can_view_all.
getGeneralDashboardSummary(userId)	L2896	Visão geral da família: patrimônio total, total receitas e despesas acumuladas, todos os membros.
getMonthlyChart(userId, months)	L2938	Série histórica de N meses para gráfico de barras (receita vs. despesa). Default 6 meses.
getCategoryChart(userId, month, year)	L2960	Agrupamento de gastos por categoria do mês para gráfico de donut.
getBudgets(userId, month, year)	L2984	Lista orçamentos do mês com gasto real calculado por categoria. Suporta can_view_all.
setBudget(data)	L3020	Define ou atualiza limite de orçamento para categoria/mês/ano via INSERT OR REPLACE.
getGoals(userId)	L3030	Lista metas com progresso atual (current_amount) e percentual.
createGoal / updateGoal / deleteGoal	L3034	CRUD de metas financeiras.
addGoalDeposit({goal_id, amount, note, date})	L3057	Registra aporte em meta e atualiza current_amount atomicamente.

Relatórios, Família e Exportação

MÉTODO	LINHA	DESCRIÇÃO
getCashFlow(userId, month, year)	L3066	Fluxo de caixa: agrupa transações por data e calcula saldo acumulado diário.
getPatrimony(userId)	L3079	Patrimônio: soma saldos de todas as contas (exceto crédito), histórico de 12 meses para gráfico.
logEvent(eventType, message, familyId)	L3120	Persiste evento de auditoria em server_logs. Chamado internamente em ações críticas.
getServerLogs() / getFamilyLogs(familyId)	L3128	Recupera logs de auditoria (todos ou filtrado por família).
getFamilies / createFamily / updateFamily / deleteFamily	L3136	CRUD de famílias. deleteFamily faz cascata completa: usuários, contas, metas, transações, recorrentes.
checkFamilyName(name)	L3212	Verifica se nome de família já existe (usado no wizard de cadastro).
backup(destPath)	L3239	Copia o arquivo .db para destPath usando API nativa do better-sqlite3.
exportMyData(userId)	L3307	Exporta todos os dados pessoais do usuário (LGPD). Inclui contas, categorias, recorrentes, transações, metas, membros da família.
exportFullJson(userId)	L3425	Exportação completa de todos os dados para backup JSON.

MÉTODO	LINHA	DESCRIÇÃO
<code>exportTransactionsCsv({userId, month, year, type})</code>	L3462	Gera CSV de transações mensal ou anual com cabeçalho em português.

Módulos Auxiliares do Banco

ARQUIVO	CLASSE/FUNÇÃO	DESCRIÇÃO
src/database/session-repo.js	SessionRepository	ensureTable() : cria tabela sessions se necessário. saveSession(token, user) : persiste sessão com TTL de 7 dias + limpeza de expiradas. getSession(token) : valida TTL e retorna sessão. deleteSession(token) : remove sessão. cleanupExpired() : remove todas as sessões vencidas.
src/database/crypto-utils.js	encryptField / decryptField	encryptField(plainText) : AES-256-GCM com IV aleatório de 12 bytes. Retorna base64 (IV+AuthTag+Encrypted). decryptField(payload) : detecta legado (não-base64) e retorna as-is. Em falha de deciptação, também retorna as-is. Chave: env <code>DATA_ENCRYPTION_KEY</code> (64 hex chars). Em dev usa hash derivado de string fixa.

🎨 Capítulo 7 — app.js — Frontend (6.413 linhas)

src/renderer/app.js · Arquivo monolítico com toda a lógica da UI

Estado Global e Utilitários Base

FUNÇÃO/OBJETO	LINHA	DESCRIÇÃO
State (objeto global)	L~1	Estado em memória: currentUser, currentMonth, currentYear, currentPage, permissions, accounts, categories. Atualizado por <code>startApp</code> e <code>navigate</code> .
Modal (objeto)	L287	Controller do modal central: <code>Modal.open(title, bodyHTML, wide, isSettings)</code> e <code>Modal.close()</code> . Um único modal reutilizado por todos os formulários.
renderAvatarHtml(user, size)	L266	Gera HTML de avatar: SVG pré-definido (AVATARS map) ou inicial colorida geométrica.
toast(message, type)	L276	Notificação temporária (3.2s). Tipos: success/error/warning/info. Auto-remove com animação.
navigate(page)	L303	Troca de aba/página com verificação de permissão. Se página restrita, redireciona para primeiro menu permitido. Chama <code>renderPage</code> .
renderPage(page)	L322	Dispatcher: mapeia página para função de render (dashboard, recurring, accounts, budget, goals, reports, settings, families).
buildPeriodSelector(onUpdate)	L328	Constrói seletor de mês/ano (< label >) com callbacks de navegação. Atualiza State.currentMonth/Year.
bankLogo(bank, size)	L343	Retorna HTML de logo de banco por nome (mapa de ícones SVG inline por banco).
fmtCompetence(compStr)	L2795	Formata string 'YYYY-MM' em 'Mês/YYYY' em português. Usado para exibição do mês de referência (competência).

Dashboard

FUNÇÃO	LINHA	DESCRIÇÃO
buildDonut(pct, color, size)	L572	Gera SVG de donut percentual para cards de orçamento e metas.
buildCreditDonut(spent, limit, size)	L590	Donut para uso de crédito. Muda de cor (verde → amarelo → vermelho) conforme percentual.
renderDashboard()	L639	Página principal: chama <code>dashboard:getSummary</code> , renderiza cards de conta, gráfico de categorias, metas, resumo e period selector.
renderCreditCardWidget(acc, spent)	L1023	Card do cartão de crédito com donut, valor gasto, limite e acesso à fatura.
renderDebitAccountWidget(acc)	L1098	Card interativo de conta corrente/poupança com saldo e ações.
renderDebitAccountStaticWidget(acc)	L1124	Versão simplificada (sem botões) para exibição em modo de leitura.
renderDashboardGoalItem(goal)	L1150	Item de meta no dashboard: nome, barra de progresso, valor atual vs. alvo.
setupCategoryInteractiveChart(wrapperElementId, chartStateKey, txs)	L1174	Configura gráfico de barras interativo de categorias usando Chart.js. Atualiza ao clicar nos filtros.

FUNÇÃO	LINHA	DESCRIÇÃO
chartOptions(type)	L1410	Retorna configuração de options do Chart.js por tipo (bar, line, doughnut). Dark mode integrado.

Recorrentes e Faturas

FUNÇÃO	LINHA	DESCRIÇÃO
renderRecurring()	L1454	Página de recorrentes: busca recurring:getAll, recurring:getMonthly e invoices:getMonthly. Renderiza abas (Despesas Fixas, Receitas, Avulsos, Faturas).
setupDragAndDrop(container, isRecurring)	L1610	Configura drag-and-drop HTML5 para reordenar itens. Chama recurring:updatePositions ou transactions:updatePositions ao soltar.
renderRecurringList(container, items, monthlyTxs, type, accounts, categories)	L1659	Renderiza lista de recorrentes com status de pagamento rico: badge de referência (Ref: Mês/Ano), badges de pago antecipado/atrasado com desconto/juros, botões de ação.
renderAvulsosList(container, txs, accounts, categories, tabType)	L1935	Renderiza lançamentos avulsos com referência de competência, status de pagamento, valor base + juros/descontos. Suporta filtro por tipo de aba.
renderInvoicesList(container, invoices, accounts)	L2098	Renderiza faturas de cartão com período de fechamento, valor, status e botões: Pagar, Renegociar, Reabrir.
confirmReopenInvoice(inv)	L2200	Modal de confirmação para reabrir fatura paga. Chama invoices:reopen .
openRenegotiateInvoiceModal(inv, accounts)	L2268	Modal complexo de renegociação: define número de parcelas, entrada, conta de débito, e chama invoices:renegotiate .
openPayInvoiceModal(inv, accounts)	L2481	Modal de pagamento de fatura: conta de débito, data de pagamento, juros/desconto. Chama invoices:pay .
loadAvulsos(container, accounts, categories, tabType)	L2571	Carrega e renderiza transações avulsas via transactions:getAll com filtros do período atual.

Modais de Criação e Edição

FUNÇÃO	LINHA	DESCRIÇÃO
openRecurringModal(item, accounts, categories, type)	L2577	Modal de criação/edição de recorrentes. Campos: nome, valor, categoria, conta, dia de vencimento, Mês de Referência (competência) , Mês de Vencimento , número de parcelas, prioridade. Salva via recurring:create ou recurring:update .
openEditMonthTransactionModal(tx, item, accounts, categories, type)	L2807	Modal de edição de uma ocorrência mensal específica (não edita o item recorrente, apenas a transação do mês). Suporta edição de valor, conta e data de competência.
showDidacticFeedback(data)	L2909	Exibe painel didático de feedback após ações financeiras: mostra impacto no orçamento, comparativo mensal e dica personalizada.
openAvulsoModal(accounts, categories, tx, defaultType)	L2949	Modal de lançamento avulso (despesa/receita única). Campos: descrição, valor, categoria, conta, data, Mês de Referência (competência) , tipo. Salva via transactions:create ou transactions:update .
openPaymentDateModal(txId, currentDate, onComplete)	L3147	Modal para marcar transação como paga: data de pagamento real, juros ou desconto. Chama transactions:togglePaidWithDate . Verifica se foi antecipado ou atrasado vs. data de vencimento.

Contas, Orçamento e Metas

FUNÇÃO	LINHA	DESCRIÇÃO
renderAccounts()	L3309	Página de contas: lista todas as contas do usuário (e família se permitido), cartões de crédito em destaque, ações de edição/exclusão/transferência.
openAccountModal(acc)	L3488	Modal de criação/edição de conta. Campos dinâmicos por tipo (ex: limite e fechamento só para crédito). Chama <code>accounts:create</code> ou <code>accounts:update</code> .
openTransferModal(accounts)	L3651	Modal de transferência entre contas. Seleciona origem, destino, valor e data. Chama <code>accounts:transfer</code> .
renderBudget()	L3682	Página de orçamento: lista categorias com limite e gasto real do mês. Donut por categoria. Botão para definir limite.
openBudgetModal(cats, prefillCatId, prefillAmt)	L3760	Modal de orçamento por categoria. Pré-preenche se prefill fornecido. Chama <code>budgets:set</code> .
renderGoals()	L3792	Página de metas: lista metas com barra de progresso, prazo, opção de aporte.
openGoalModal(goal)	L3840	Modal criação/edição de meta: nome, ícone, valor alvo, prazo. Chama <code>goals:create</code> ou <code>goals:update</code> .
openGoalDepositModal(goalId, goals)	L3886	Modal de aporte: valor e nota. Chama <code>goals:addDeposit</code> .

Relatórios e Configurações

FUNÇÃO	LINHA	DESCRIÇÃO
renderReports()	L3908	Página de relatórios: gráfico histórico mensal (receita vs. despesa), fluxo de caixa diário, patrimônio acumulado.
renderSettings()	L3980	Placeholder — Settings é aberto via modal, não via página.
openSettingsModal(activeTab)	L3984	Modal de configurações com 3 abas: Perfil, Backup e LGPD. Chama <code>bindProfileTabEvents</code> , <code>bindBackupTabEvents</code> , <code>bindLgpdTabEvents</code> .
bindProfileTabEvents(categories, users)	L4507	Aba Perfil: edição de dados pessoais, avatar, categorias, membros da família, tema.
bindBackupTabEvents(capitalizedMonth)	L4673	Aba Backup: exportar Excel/JSON/CSV, restaurar banco. Usa <code>backup:exportExcel</code> , <code>backup:exportJson</code> , <code>backup:exportCsv</code> .
bindLgpdTabEvents()	L4821	Aba LGPD: exportar dados pessoais (<code>auth:exportMyData</code>), excluir conta (<code>auth:deleteSelf</code>).
openCategoryModal(categories, editCategory)	L4923	Modal de categoria: nome, ícone emoji, cor, tipo (receita/despesa).

Login, Cadastro e Controle de App

FUNÇÃO	LINHA	DESCRIÇÃO
initLoginScreen()	L4973	Tela de login: formulário, validação, <code>auth:login</code> , recuperação de senha, exibição de usuários cadastrados com avatars.
showWizardFamilyChoiceModal(familyName, ownerName, onJoin, onNew)	L5136	Modal do wizard: usuário escolhe entre entrar em família existente ou criar nova.
openSignUpWizard()	L5189	Wizard multi-passo de cadastro: Passo 1 (nome/username/senha), Passo 2 (dados pessoais), Passo 3 (família), Passo 4 (termos). Orquestra todos os passos e chama <code>auth:register</code> .
setupWizardMasksAndValidators()	L5212	Configura máscaras de CPF, telefone e validações inline do wizard.

FUNÇÃO	LINHA	DESCRIÇÃO
updateSignUpWizardUI()	L5328	Atualiza visual do wizard (barra de progresso, habilitação de botões) conforme passo atual.
openRegisterModal()	L5546	Modal simples de cadastro rápido (alternativa ao wizard para usuários já logados adicionando membro).
openEditUserModal(user)	L5569	Modal de edição completa de usuário: dados pessoais, CPF, email, avatar, permissões granulares por menu. Chama <code>auth:updateUser</code> e <code>permissions:update</code> .
applyNavigationPermissions()	L5899	Oculta/exibe itens do menu lateral baseado nas permissões <code>allow_*</code> do usuário.
startApp(user)	L5913	Bootstrap pós-login: carrega State, contas, categorias, permissões, aplica tema, inicializa navegação. Ponto central de controle do app.
checkImpersonation()	L6002	Verifica se ADM está operando como outro usuário (modo impersonação). Exibe banner de aviso.
openLanConnectModal()	L6095	Modal para conectar dispositivos na rede local: exibe IP e QR code para acesso mobile.

Modo Caçula e ADM de Famílias

FUNÇÃO	LINHA	DESCRIÇÃO
renderCaculaDashboard(page)	L6136	Dashboard simplificado para perfil "Caçula": exibe resumo simples do mês e botão de adicionar despesa rápida. Não tem acesso a menus completos.
openCaculaQuickExpenseModal(accounts)	L6244	Modal simplificado de despesa rápida para o Caçula: valor, descrição, conta. Interface reduzida sem categorias complexas.
renderLogsInConsole(logs)	L6338	Renderiza logs de auditoria na tela ADM de famílias. Suporta filtro por tipo de evento.
renderFamilies()	L6368	Página de administração de famílias (ADM Geral): lista famílias, usuários, estatísticas, logs. Acesso restrito a <code>isSystemAdmin</code> .
openAdmNewFamilyModal()	L6558	Modal de criação de nova família (ADM). Campos: nome, quota de usuários e contas.
openAdmEditFamilyModal(familyId)	L6646	Modal de edição de família existente: gerencia membros, quotas e pode deletar família com cascata.

LGPD e Tema

FUNÇÃO	LINHA	DESCRIÇÃO
initLgpdModals()	L6906	Inicializa modais de LGPD na tela de login: exibe termos de uso, coleta aceite com timestamp e versão.
setAppTheme(themeName)	L6965	Aplica tema ao body via classe CSS. Temas: dark (padrão), light, forest, ocean, sunset.
initThemeSwitcher()	L6979	Inicializa seletor de temas nas configurações e carrega tema salvo do usuário via <code>settings:get</code> .

🔧 Capítulo 7b — Módulos JS Auxiliares do Frontend

src/renderer/js/api.js — Proxy de API Web

Módulo ES6 exportado. No Electron, retorna `window.api` diretamente. No navegador web, cria um `Proxy` que intercepta qualquer chamada de método e a traduz para `POST /api/rpc` com Bearer Token do localStorage. Também lida com `isWebDownload` (base64 → download automático no browser).

FUNÇÃO/OBJETO	LINHA	DESCRIÇÃO
<code>SESSION_KEY = 'ff_session_token'</code>	L1	Chave do localStorage para armazenar o Bearer Token da sessão web.
<code>export const api</code>	L3	Proxy universal: qualquer <code>api.namespace.method(...args)</code> vira <code>POST /api/rpc {channel: 'namespace:method', args}</code> .
<code>isWebDownload handler</code>	L35	Se a resposta tiver <code>isWebDownload: true</code> , decodifica o base64 e dispara download automático pelo browser.

src/renderer/js/state.js — Estado Global (módulo)

PROPRIEDADE/MÉTODO	DESCRIÇÃO
<code>currentUser</code> , <code>activeTab</code> , <code>selectedMonth</code> , <code>selectedYear</code>	Estado básico do app
<code>accounts[]</code> , <code>categories[]</code> , <code>recurringItems[]</code> , <code>transactions[]</code>	Caches de dados em memória
<code>setUser(user)</code>	Define o usuário autenticado
<code>setPeriod(month, year)</code>	Define o período selecionado
<code>setActiveTab(tab)</code>	Define a aba ativa

src/renderer/js/formatters.js — Formataadores

FUNÇÃO	DESCRIÇÃO
<code>formatCurrency(value)</code>	Formata número para BRL (R\$ X.XXX,XX) via Intl.NumberFormat
<code>formatDate(dateString)</code>	Converte ISO (YYYY-MM-DD) para DD/MM/YYYY
<code>formatCPF(cpf)</code>	Aplica máscara XXX.XXX.XXX-XX ao CPF
<code>sanitizeHtml(str)</code>	Escapa <code><</code> <code>></code> <code>&</code> <code>"</code> para prevenção de XSS

🔍 Capítulo 8 — Auditoria Completa de Código

⚠️ 8.1 — Duplicatas Críticas (código que polui e cria riscos)

🔥 BLOCO DUPLICADO em db.js (L3270–L3420)

Um bloco de 150 linhas ao final do arquivo **redeclara 6 métodos** que já existem anteriormente na mesma classe `AppDatabase`. Isso cria risco de: (1) dessincronização ao editar um mas não o outro, (2) comportamento inesperado dependendo de qual declaração "vence", (3) desperdício de memória.

MÉTODO	1ª DECLARAÇÃO (MANTER)	2ª DECLARAÇÃO (REMOVER)	EXISTE 3ª?
<code>getUserById</code>	L3271	L3372 ✖	Não
<code>checkAccountFamily</code>	L3279	L3380 ✖	Não
<code>checkTransactionFamily</code>	L3284	L3389 ✖	Não
<code>checkCategoryFamily</code>	L3289	L3398 ✖	Não
<code>checkGoalFamily</code>	L3297	L3407 ✖	Não
<code>checkRecurringFamily</code>	L3302	L3416 ✖	Não

⚠️ `loadTab` redefinida em app.js

A função `loadTab` é declarada em `app.js:L1521` e **redefinida silenciosamente** em `app.js:L3926`. A segunda declaração sobrescreve a primeira no escopo global. Isto pode causar comportamento incoerente dependendo de qual lógica deveria prevalecer.

👤 8.2 — Código Oculto / Funcionalidades Sem UI (precisam de decisão)

FUNCIONALIDADE	ONDE ESTÁ IMPLEMENTADA	STATUS NA UI	DECISÃO RECOMENDADA
<code>backup:restore</code> (Restaurar banco via upload)	<code>core.js:L389</code> <code>preload.js:L91</code>	SEM BOTÃO NA UI A lógica existe: valida o arquivo SQLite, faz backup do atual, restaura o novo. O preload expõe o canal. Mas nenhum botão na tela de configurações o chama.	✅ Criar botão "Restaurar Backup" na aba Backup das Configurações
<code>server:getInfo</code>	<code>preload.js:L11</code>	SEM USO APARENTE Canal público exposto no preload mas sem handler visível no buildHandlers de core.js e sem uso em app.js.	🔍 Investigar se é usado por código externo ou remover do preload
<code>renderLogsInConsole(logs)</code>	<code>app.js:L6338</code>	ATIVA COM RESTRIÇÃO Função implementada e usada na página de ADM de Famílias. Visível apenas para <code>isAdmin=1</code> .	✅ Manter como está — uso correto e restrito
<code>state.js</code> e <code>formatters.js</code>	<code>src/renderer/js/state.js</code> <code>src/renderer/js/formatters.js</code>	SUBUTILIZADOS Exportados como módulos ES6 mas app.js usa seu próprio objeto State interno. Estes módulos parecem ser de uma refatoração iniciada e não concluída.	📁 Decidir: concluir a migração para módulos ES6 ou remover os arquivos

🛡️ 8.3 — Funcionalidades Condicionais (ativas mas visíveis por perfil)

Estas funcionalidades estão 100% implementadas e funcionando, mas só aparecem para determinados perfis:

FUNCIONALIDADE	CONDIÇÃO DE ATIVAÇÃO	FUNÇÕES ENVOLVIDAS
Dashboard do Caçula	<code>profile_type === 4</code>	<code>renderCaculaDashboard · openCaculaQuickExpenseModal</code>
Gestão de Famílias (ADM)	<code>is_system_admin === 1</code>	<code>renderFamilies · openAdmNewFamilyModal · openAdmEditFamilyModal</code>
Logs de Auditoria	<code>profile_type ∈ {1, 2}</code>	<code>renderLogsInConsole</code>
Backup do banco (nativo)	<code>is_system_admin === 1</code> ou <code>profile_type === 1</code>	<code>main.js:backup:export</code>
Modo impersonação	Sessão ADM ativa + localStorage com userId diferente	<code>checkImpersonation</code>
Permissões granulares por menu	Definidas pelo Responsável via <code>openEditUserModal</code>	<code>applyNavigationPermissions</code>

🔧 8.4 — Scripts de Manutenção (scripts/)

Estes arquivos **não fazem parte do app em produção**. São ferramentas de manutenção manual para o desenvolvedor. Não são importados nem executados automaticamente.

SCRIPT	PROPÓSITO	QUANDO USAR
<code>scripts/db-diag.js</code>	Diagnóstico do banco de dados	Verificar integridade do SQLite em produção sem abrir o app
<code>scripts/generate-keys.js</code>	Gerador de <code>DATA_ENCRYPTION_KEY</code>	Configuração inicial de produção — gera chave hex de 64 chars para o <code>.env</code>
<code>scripts/migrate-encrypt-cpf.js</code>	Migração de CPF para criptografia	One-time: criptografa CPFs existentes em plaintext após habilitar encriptação
<code>scripts/query-users.js</code>	Consulta manual de usuários	Verificar dados de usuário no banco diretamente, sem UI
<code>scripts/test-financial-engine.js</code>	Suite de testes do motor financeiro	Validar lógica de cálculos financeiros antes de grandes mudanças no <code>db.js</code>

Capítulo 9 — Plano de Ação e Recomendações de Engenharia

Prioridade ALTA — Fazer antes de qualquer nova feature

AÇÃO	ARQUIVO	IMPACTO
Remover bloco duplicado L3370-L3420 em db.js	src/database/db.js	Elimina 50 linhas desnecessárias, reduz risco de bug silencioso
Unificar as duas declarações de <code>loadTab</code> em app.js	src/renderer/app.js	Elimina comportamento imprevisível de override silencioso
Criar botão "Restaurar Backup" na aba Backup (Configurações)	src/renderer/app.js	Ativa funcionalidade 100% implementada que está oculta

Prioridade MÉDIA — Dívida técnica a endereçar

AÇÃO	ARQUIVO	IMPACTO
Decidir destino de <code>state.js</code> e <code>formatters.js</code> (migrar ou remover)	src/renderer/js/state.js	Clareza arquitetural — refatoração incompleta
Investigar e remover/documentar <code>server:getInfo</code>	src/preload.js	Remover superfície de ataque desnecessária
Modularizar <code>app.js</code> em arquivos por domínio (accounts, recurring, etc.)	src/renderer/app.js	Arquivo de 6413 linhas é difícil de manter. Separar em módulos ES6.

Boas Práticas para Novas Implementações

1. **Ao criar novo método no db.js:** Adicionar dentro da classe, nunca após `module.exports`
2. **Ao criar novo canal RPC:** Adicionar em `buildHandlers` + `createOwnershipChecks` + `preload.js` (três lugares sempre)
3. **Ao criar nova migration:** Incrementar o número (atualmente até 22) e usar try/catch idempotente
4. **Ao criar função no app.js:** Verificar se já existe função similar antes de criar nova. Documentar chamadas de API que usa.
5. **Ao modificar cálculos de data/competência:** Testar com `scripts/test-financial-engine.js`
6. **Ao adicionar campo sensível (CPF, etc.):** Usar `encryptField` / `decryptField` de `crypto-utils.js`